

Kolokwium 1 - Java

Imię:

Nazwisko:

Na napisanie testu jest 90 minut. Każde poprawnie rozwiązane zadanie to jeden punkt. Każde niepoprawnie rozwiązane zadanie to zero punktów (dotyczy to również zadań z wielokrotnym wyborem).

1. Poniższy kod kompiluje się i uruchamia bez problemów. Jaki jest wynik jego wykonania?

```
class One {  
    { System.out.print("A"); }  
    static { System.out.print("B"); }  
    One() { System.out.print("C"); }  
}  
class Two extends One {  
    { System.out.print("D"); }  
    static { System.out.print("E"); }  
    Two() { System.out.print("F"); }  
}  
public class InhTest {  
    { System.out.print("G"); }  
    static { System.out.print("H"); }  
    InhTest() { System.out.print("J"); }  
    public static void main(String[] args) { new Two(); }  
}
```

2. Poniższy kod kompiluje się i uruchamia bez problemów. Jaki jest wynik jego wykonania?

```
class A{}  
class B extends A{}  
class C extends B{}  
public class Polimorfizm {  
    static void mimi(A x){System.out.print("A");}  
    static void mimi(B x){System.out.print("B");}  
    static void mimi(C x){System.out.print("C");}  
    public static void main(String[] args) {  
        B x = new C();  
        mimi(x);  
        mimi(null);  
    }  
}
```

3. Poniższy kod kompiluje się i uruchamia bez problemów. Jaki jest wynik jego wykonania?

```
public class Seven {
    static int seven = 7;
    public static void main(String [] a) { new Seven(); }
    Seven() { System.out.print(++seven); }
    static {
        new Seven();
        System.out.print(--seven);
    }
    { System.out.print(seven); }
```

4. Poniższy kod kompiluje się i uruchamia bez problemów. Jaki jest wynik jego wykonania?

```
public class Concat {
    public static void main(String[] args) {
        String s1 = "A";
        String s2 = "B";
        String s3 = s1.concat("C");
        s1.concat(s2);
        System.out.println(s1);
    }
}
```

5. Jeśli klasa A posiada metodę
`public Object t(String s) { return "A"; }`
to klasa B dziedzicząca po A może posiadać metodę:
- a. `public String t(Object o) { return "B"; }`
 - b. `String t(String s) { return "B"; }`
 - c. `public Object t(String s) { return "B"; }`
 - d. `public String t(String s) { return "B"; }`
 - e. `Integer t(String s) { return 2; }`
 - f. `public Integer t(String s) { return 2; }`
 - g. `public Integer t(Integer i) { return 2; }`

6. Dany jest poniższy kod

```
public class Outside {
    private class Inside {Inside(){} }
    public static void makeObject() {
        ###
    }
}
```

Zaznacz linie, które po wstawieniu w miejsce oznaczone `###`, skompilują się poprawnie.

- a. new Inside();
- b. Outside.new Inside();
- c. new Outside().new Inside();
- d. new Outside.new Inside();
- e. new Inside(new Outside());
- f. żadna z powyższych.

7. Dany jest poniższy kod

```
public class Outside {
    private static class Inside {Inside(){} }
    public static void makeObject() {
        ###
    }
}
```

Zaznacz linie, które po wstawieniu w miejsce oznaczone ###, skompilują się poprawnie.

- a. new Inside();
- b. Outside.new Inside();
- c. new Outside().new Inside();
- d. new Outside.new Inside();
- e. new Inside(new Outside());
- f. żadna z powyższych.

8. Klasa A<T> jest zdefiniowana następująco:

```
class A<T> { T id(T t) { return t; } }
```

zaznacz linie które skompilują i wykonają się poprawnie:

- a. A<Integer> a = new A<Integer>(); Integer i = a.id(7);
- b. A<? extends Integer> b = new A<Integer>(); Integer j = b.id(7);
- c. A<? super Integer> c = new A<Integer>(); Object k = c.id(7);
- d. A<?> d = new A<Integer>(); Object l = d.id(null);

9. Dany jest poniższy kod

```
public class House<T> {
    static <T> void live(House<T> a) {}
    class Chair<S extends House<?>> {
        S sit() { return null; }
    }
    static {
        ###
    }
}
```

Zaznacz linie, które po wstawieniu w miejsce oznaczone ###, skompilują się poprawnie.

- a. `new House<Integer>().new Chair<Integer>().sit();`
- b. `new House<Number>().new Chair<Integer>().sit();`
- c. `new House<Integer>().new Chair<Number>().sit();`
- d. `new House<Integer>().new Chair<House<Integer>>().sit();`
- e. `new House<Number>().new Chair<House<Integer>>().sit();`
- f. `new House<Integer>().new Chair<House<Number>>().sit();`
- g. `new Chair<House<Integer>>().sit();`
- h. `new House<Integer>() { { this.live(new House<String>()); } };`
- i. `new House<Integer>() { { this.live(new House<Integer>()); } };`

10. Dany jest poniższy kod

```
public class Zadanie<T> {
    public <S extends T> void test(Collection<? extends T> a,
        Collection<S> b, Collection<T> c){
        ###
    }
}
```

Zaznacz linie, które po wstawieniu w miejsce oznaczone ###, skompilują się poprawnie.

- a. `a.addAll(a);`
- b. `a.addAll(b);`
- c. `a.addAll(c);`
- d. `b.addAll(a);`
- e. `b.addAll(b);`
- f. `b.addAll(c);`
- g. `c.addAll(a);`
- h. `c.addAll(b);`
- i. `c.addAll(c);`

11. Zdefiniowana jest następująca metoda:

```
static boolean test(Integer a, Integer b) {
    return a == b;
}
```

zaznacz zdania prawdziwe:

- a. Funkcja `test` zwraca `true` wtedy i tylko wtedy, gdy zmienne `a` i `b` wskazują na tę samą instancję klasy `Integer`.
- b. Funkcja `test` zwraca `true` wtedy i tylko wtedy, gdy wartości liczbowe reprezentowane przez zmienne `a` i `b` są równe.
- c. Funkcja `test` zwraca `true` wtedy i tylko wtedy, gdy zmienna `a` wskazuje na pewien obiekt oraz `a.equals(b)` zwróciłoby `true`.
- d. Jeśli zmienne `x` i `y` wskazują na obiekty klasy `Integer` reprezentujące tę samą wartość liczbową, wywołanie `test(x, y)` na pewno zwróci `false`.
- e. Jeśli zmienne `x` i `y` wskazują na obiekty klasy `Integer` reprezentujące tę samą wartość liczbową, wywołanie `test(x, y)` na pewno zwróci `true`.
- f. Jeśli zmienne `x` i `y` wskazują na obiekty klasy `Integer` reprezentujące tę samą wartość liczbową, wywołanie `test(x, y)` może zwrócić `true` lub `false`.

12. Zdefiniowane są następujące metody:

```
public static void array(Object[] o) { }  
public static void listObj(List<Object> l) { }  
public static void listExtObj(List<? extends Object> l) { }  
zaznacz wszystkie poprawne wywołania powyższych metod.
```

- a. array(new Object[] {1,2,3});
- b. array(new Integer[] {1,2,3});
- c. listObj(Arrays.asList(new Object[] {1,2,3}));
- d. listObj(Arrays.asList(new Integer[] {1,2,3}));
- e. listExtObj(Arrays.asList(new Object[] {1,2,3}));
- f. listExtObj(Arrays.asList(new Integer[] {1,2,3}));

13. Dany jest poniższy kod:

```
public class Exceptions {  
    static class B extends RuntimeException {  
        public B() { throw this; } // 1  
    }  
    static class A extends RuntimeException {  
        public A() { throw new B(); } // 2  
    }  
    public static void main(String[] args) {  
        throw new A(); //3  
    }  
}
```

Zaznacz wszystkie poprawne stwierdzenia:

- a. program wyrzuci wyjątek metodą throw numer 1
- b. program wyrzuci wyjątek metodą throw numer 2
- c. program wyrzuci wyjątek metodą throw numer 3
- d. program wejdzie w nieskończoną pętlę

14. Dany jest poniższy kod:

```
public class ExceptionsThread {  
    static class A extends Thread {  
        public void run() { throw new RuntimeException(); }  
    }  
    public static void main(String[] args) {  
        System.out.println("Beginning");  
        new A().start();  
        System.out.println("Middle");  
        new A().run();  
        System.out.println("The End");  
    }  
}
```

Zaznacz wszystkie poprawne stwierdzenia:

- a. Program wypisze "Beginning".
- b. Program wypisze "Middle".
- c. Program wypisze "The End".
- d. Podczas działania programu zostanie wyrzucony wyjątek.

15. Dany jest poniższy kod:

```
public static void main(String[] args) {  
    Thread t1 = new Thread(new Runnable() {  
        public void run() { while(true); }  
    });  
    t1.setDaemon(true);  
    t1.start();  
    t1 = new Thread(new Runnable() {  
        public void run() { while(true); }  
    });  
    t1.run();  
    t1.setDaemon(true);  
}
```

Zaznacz wszystkie poprawne stwierdzenia:

- a. Podczas działania programu wyrzucony zostanie wyjątek.
 - b. Program będzie działać w nieskończoność.
 - c. Program zakończy się nic nie wypisując na wyjście.
16. Zaznacz wszystkie prawdziwe stwierdzenia. A to dowolna klasa, a obj - dowolny obiekt.
- a. Jeśli `obj.getClass().equals(A.class)` jest prawdą, to `(obj instanceof A)` jest prawdą.
 - b. Jeśli `(obj instanceof A)` jest prawdą, to `obj.getClass().equals(A.class)` jest prawdą.
 - c. Jeśli `obj == null`, to `(obj instanceof A)` jest fałszem.
 - d. Jeśli `obj.getClass().isAssignableFrom(A.class)` to `(obj instanceof A)` jest prawdą.
17. Zaznacz wszystkie prawdziwe zdania dotyczące interfejsów.
- a. Metody deklarowane w interfejsie nie są domyślnie ani `public`, ani `private`, ani `protected`.
 - b. Zmienne deklarowane w interfejsie są domyślnie `protected`, `static` i `final`.
 - c. Klasy mogą implementować wiele interfejsów jednocześnie.
 - d. Interfejs nie może dziedziczyć po wielu interfejsach jednocześnie.
 - e. Słowo kluczowe `extends` oznacza, że interfejs dziedziczy po innym interfejsie.
18. Czy jest możliwe zdefiniowanie dwóch interfejsów tak, żeby klasy mogły je implementować, ale nigdy równocześnie?
- a. nie
 - b. tak - podaj przykład.
19. Czy podany poniżej komparator jest dobrze zaprojektowany? Jeśli nie, to podaj uzasadnienie.
- ```
class Zadanie implements Comparator<Object> {
 public int compare(Object o1, Object o2) {
 return o1.hashCode() < o2.hashCode() ? 1 :
 o1.hashCode() == o2.hashCode() ? 0 : -1;
 }
}
```

20. W który miejscu poniższej metody obiekt zainicjalizowany w linii 1 może zostać (najwcześniej!) odśmiecony:

```
void method() {
 String a = new String("aaa"); // 1
 java.lang.ref.WeakReference<String> b =
 new java.lang.ref.WeakReference<String>(a); // 2
 a = a + 1; // 3
 a = null; // 4
}
```

- a. bezpośrednio po linii 1
- b. bezpośrednio po linii 2
- c. bezpośrednio po linii 3
- d. bezpośrednio po linii 4
- e. nigdy

Zanalizuj poniższe kody (zakładamy że wszystkie importy są w porządku, a cały kod znajduje się w pliku Zadanie.java) i:

- jeśli nastąpi błąd kompilacji pliku Zadanie.java, to wskaż w którym miejscu,
  - jeśli plik się skompiluje, ale po uruchomieniu wyrzucony zostanie wyjątek to wskaż w której linii,
  - jeśli kod się wykona bez wyjątku to podaj wynik uruchomienia klasy Zadanie.
- 21.

```
class Zadanie {
 public static void main(String[] args) {
 try {
 System.out.println(1/0);
 } catch (Exception e) {
 System.out.println("A");
 } catch (ArithmeticException e) {
 System.out.println("B");
 }
 }
}
```

22.

```
public class Zadanie {
 public static void main(String[] args) {
 try{
 throw new RuntimeException();
 }catch(RuntimeException e) {
 System.out.println("A");
 throw new Exception();
 }catch(Exception e) { System.out.println("B");
 }finally { System.out.println("C");
 }
 }
}
```

23.

```
class A<T extends Throwable> {
 A() throws T { System.out.print("A"); throw null; }
}
class B extends A<Exception> {
 B() throws RuntimeException { System.out.print("B"); }
}
public class Zadanie {
 public static void main(String[] args) throws Throwable { new B(); }
}
```

24.

```
public class Zadanie implements Booer {
 public static void main(String[] args) {} {
 new Zadanie();
 }
}
public interface Booer {
 public static class ABoo {
 static String boo() { return "BOO"; }
 }
}
```

25.

```
public class Zadanie {
 public static void main(String[] args) {
 Thread[] ar = new Thread[5];
 for (int i = 0; i < 5; i++) {
 ar[i] = new Thread() {
 public String toString() {
 return "" + Thread.currentThread().isDaemon();
 }
 };
 if (i % 2 == 0) ar[i].setDaemon(true);
 ar[i].start();
 }
 for(Thread t : ar) System.out.print(t);
 }
}
```

26.

```
public class Zadanie {
 public static void f() { throw new RuntimeException(); }
 public static void main(String[] args){
 try{
 try { f(); } finally {}
 } catch(Exception e) { System.out.print("1");}
 try{
 try { f(); } finally {return;}
 }
 }
}
```



```

 } catch(Exception e) { System.out.print("2");}
 }
}
27.
public enum Zadanie {
 ONE { void t() { System.out.println("1"); } },
 TWO { void t() { System.out.println("2"); } };
 public static void main(String[] args){
 System.out.println(ONE.getClass() == TWO.getClass());
 for(Zadanie temp : values()) temp.t();
 }
}
28.
class Zadanie {
 Zadanie(int i) throws RuntimeException {
 System.out.println("A");
 throw new RuntimeException();
 }
 Zadanie(boolean b) {
 this(1);
 System.out.println("B");
 }
 Zadanie(double d) {
 try { this(true);
 } finally { System.out.println("C"); }
 }
 public static void main(String[] args) {
 new Zadanie(2.0);
 }
}
29.
public class Zadanie {
 private static Zadanie TheOne = new Zadanie();
 private Zadanie(){ System.out.println("I am the One."); }
 public static void main(String[] args) {
 Zadanie x = new Zadanie();
 }
}
30.
public class Zadanie {
 public static void main(String[] args) {
 System.out.println("Co za wspaniałe kolokwium!");
 }
}

```